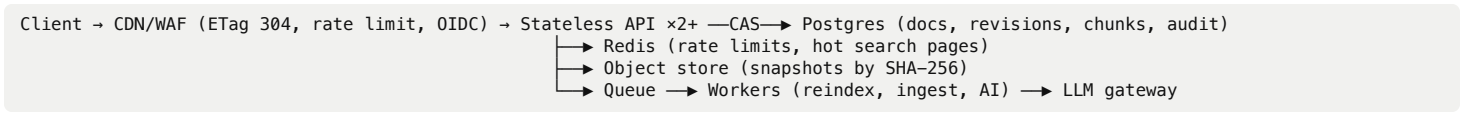
CLAUSE: production design

Where it stands: the deployed demo is open, shared, and fictional. Hosted runs on Vercel functions + Neon Postgres; local runs on SQLite.

Working assumptions (mine, not a brief): contracts around 1 MiB with a 25 MiB tail, hundreds of workspaces, up to 50 concurrent editors each, reads far above writes, 99.9% availability goal (unproven), US and EU customers.

Every number below is tagged. **Measured** means this repo on an Apple M4 laptop, Node 24/26, one process, SQLite unless a row says Postgres (laptop numbers swing ~2× with background load). **Target** means what production must hit. **Untested** means nobody has run it.



WHAT THE PROTOTYPE MEASURED

OPERATION (MS)	MEASURED	PRODUCTION TARGET	WHAT IT TELLS US
Edit engine, 1 replacement in 10 MiB	**7**	n/a	The algorithm is not the cost; a 100-change batch is 9
Full-stack edit, 10 MiB (SQLite)	**789**	n/a: Postgres refuses > 500k chars	Time is two index updates + a 10 MB snapshot + the response
Words search, 10 MiB, rare / common	**21 / 25**	p95 < 300	Index lookup is cheap once snippets are paginated (was 8,337 before the fix)
Exact search, 10 MiB, unique / many hits	**16 / 52**	p95 < 300	Trigram scan; grows with hit count
Read 1 MB doc, p95: 1 client / 8 clients	**2 / 11**	p95 < 50	ETag + `304` makes repeat reads near free
Words search 1 MB, p95: 1 / 8 clients	**2 / 173**	p95 < 300	CPU-bound single process: 77 req/s at 8 clients
Exact search 1 MB, p95: 1 / 8 clients	**5 / 435**	p95 < 300 (**missed at 8**)	Needs chunked index + replicas
Edit 1 MB, p95: 1 client / 8 writers	**61 / 879**	p95 < 500 (**missed at 8**)	One SQLite writer: throughput plateaus at ~19 edits/s
Postgres at its 500k cap, 8 clients, warmed: p95 read / words / exact / edit	**20 / 129–158 / 240–246 / 437–443**	50 / 300 / 300 / 500 (**all met**)	27 edits/s, 2 runs. A freshly loaded DB was far slower at first (words p95 4.4 s)
LLM judge call	**~2,400** median	hard deadline 25,000	Always async/advisory; the whole benchmark run cost $1.52

Sizing arithmetic (from the assumptions, not a load test): 500 concurrent editors at one edit per 15 s is ~33 edits/s. One SQLite writer tops out near 19/s and one Postgres instance measured 27/s at the 500k cap, so production needs headroom: per-document row locks and at least two API replicas. The p95 edit < 500 ms target at 1 MiB cannot be tested on Postgres until the 500k cap is lifted.

DECISIONS BY TOPIC

TOPIC	DESIGN	NUMBER	TRADE-OFF
Compute	Stateless API, ≥2 replicas across zones; workers separate	scale on p95 latency/CPU; workers on queue age (alert > 5 min)	More instances to run vs. one writer bottleneck
Storage	Postgres: metadata, current text ≤1 MB, chunks. Snapshots in object storage by hash; full every ~20 revisions + deltas	10 MB doc ≈ 10 MB per edit today	Restore replays ≤20 deltas
Search	`tsvector` + `pg_trgm` GIN on the whole document today; per-paragraph chunks with offsets only to lift the cap; rank, then snippet only the page	`tsvector` caps ~1 MB; whole-row snippet was 8.3 s on 10 MiB	Chunks (prototype): edits ~4× cheaper, searches 9–17× slower, index ~5× bigger
Index freshness	Same transaction for small edits; outbox + worker for large	`indexedVersion` returned; lag alert p95 > 2,000 ms	Bounded staleness for huge docs only
Caching	Versions are immutable: cache by `(id, version/ETag)` at the edge; Redis only for rate limits and hot search pages keyed by `(query, indexedVersion)`	read p95 11 ms, repeat reads `304`	Stale search page until `indexedVersion` moves
Concurrency	Version compare-and-swap, `If-Match`, idempotency keys; no auto-merge	stale write → `409` / `412` in 14 ms on 10 MiB (measured)	Users retry; CRDTs would change the model
Batching	A PATCH batch is the atomic unit (capped; 5,000 now); reindex/ingest batched in workers	100 edits = 9 ms engine	Large batch holds the document lock longer
CI/CD	Previews → typecheck, lint, coverage floors, both adapters, browser + a11y, fuzz weekly, audit; expand → migrate → contract; progressive rollout, one-click rollback	engine ≥98% lines, API ≥90%	Slower pipeline vs. fewer bad deploys
Security	OIDC/SSO + roles; `tenant_id` + Postgres row-level security; TLS, KMS at rest, per-tenant data keys; audit stream of actor, version, hashes (never text)	demo has none (open, labelled)	Per-tenant keys add key management; SOC 2/GDPR need audited evidence
Resilience	Multi-zone Postgres failover, PITR, restore drill, idempotent workers + DLQ	RPO ≈ 0 in region, RTO < 5 min, DR RPO ≤ 5 min (targets)	One writer per document, so no active-active
Observability	Latency by document-size bucket, conflict rate, index lag, queue age, DB lock waits, AI cost per tenant; logs/traces carry IDs and hashes only	page: edit p95 > 500 ms for 5 min, 5xx > 1%, queue age > 5 min	Metrics cardinality vs. visibility
Cost & regions	Cap document/batch size; right-size from p95; per-tenant AI budget (today 20 suggestions / 60 checks per IP per hour); regional cells for residency	$ per 1k edits, $ per GB-month of revisions	Cells duplicate infra; multi-region writes deferred

RISKS AND FIRST MEASUREMENTS

1. Postgres is measured only up to its 500,000-character cap; raising it needs chunked indexes (a `tsvector` tops out near 1 MB). Repeat `pnpm load --storage=postgres` after.
2. Exact search and edit p95 miss their targets at 8 clients on SQLite; on Postgres at the cap they are met when warmed, with little edit headroom (437 of 500 ms). Locks and chunked indexes for larger documents are unproven.
3. No tenancy today. Nothing here is customer-ready until authentication, row-level security, and audit are in; SOC 2 and GDPR are evidence and review, not architecture.

Raw numbers: `bench/performance`, `pnpm load`. Longer alternatives: appendix.